MediChain: Architecture (v1.2, revised draft)

Oct 14, 2026 · @Abc

Purpose and scope

This file is the complete architecture of MediChain: the main architecture first, then one architecture for each feature or mechanism. Everything targets one local Hardhat network on one developer machine. There is no testnet or production deployment in this phase. The file explains the design. It adds no decisions of its own, and where it shows a change it applies a decision from PRD v1.6.

Item	Value
Version	1.2, revised draft. Decisions D10 to D23 are accepted.
Date	14 October 2026 (v1.1 was dated 12 October 2026)
Based on	TRD v1.9, PRD v1.6, App Flow v1.7, Backend Schema v1.2, errata 1 to 19, and the team's four module files
Team	Group 02, Department of CSE, BGS College of Engineering and Technology, Bengaluru. Course: Project Phase-I (BCS685), confirm the label is still right

The TRD supersedes the errata file, which stays in the repository for history only. Where documents differ, the TRD wins on behavior, and the Backend Schema wins on exact field and function shapes. The Architecture follows both.

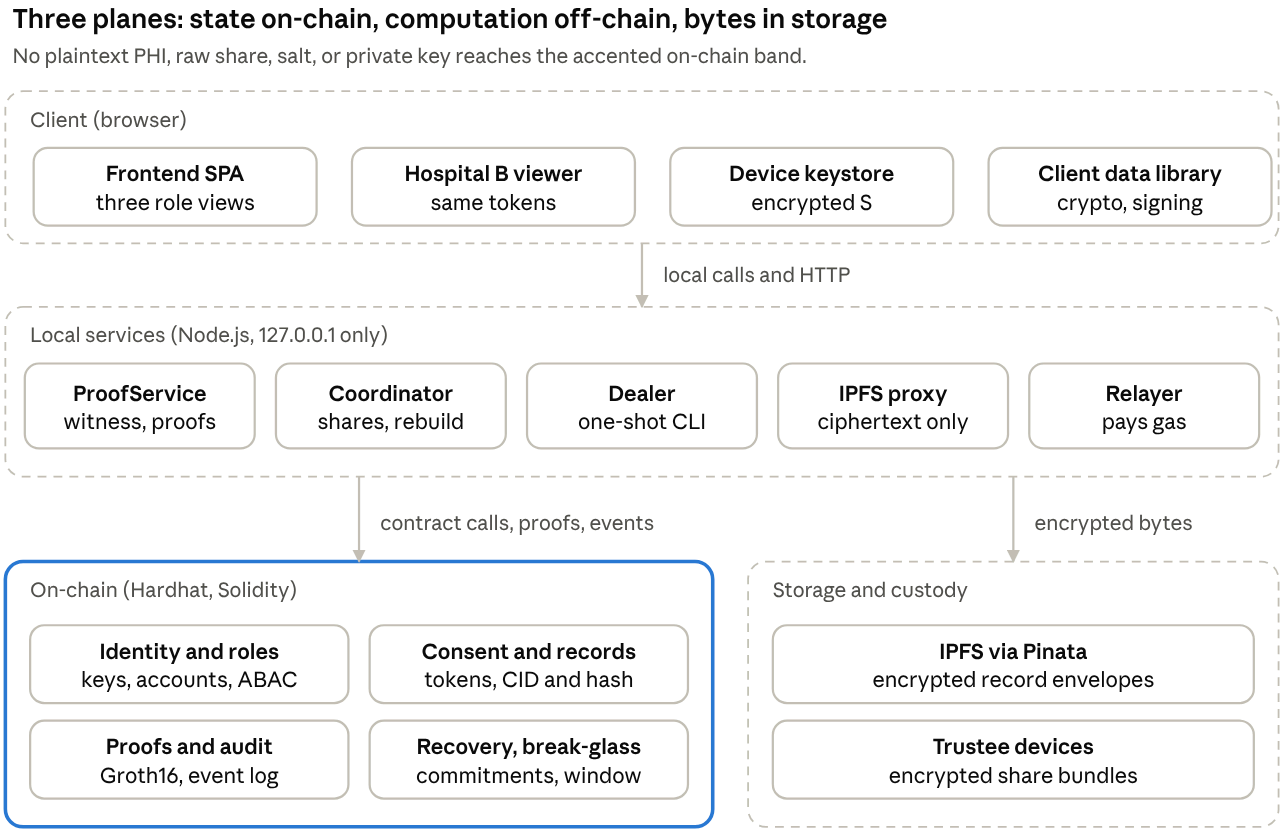
How it is organized

- **Main architecture.** The three planes, the component catalog and dependencies, what each party can see, cross-cutting design, the deployment topology, and the decisions and limits that shape everything.
- **One architecture per mechanism.** Identity and keys, the data layer, consent and access control, Tier 1 recovery, break-glass and Tier 2, zero-knowledge claims, and the audit trail. A last section maps each screen to the architecture it touches.

- **Drawings.** The planes, the key hierarchy, and the break-glass state machine are drawn. Straight-run flows, such as the record write path, are numbered steps.

The three planes

The rule that shapes everything: nothing that touches plaintext PHI, a raw secret share, a salt, or a private key ever crosses into the on-chain plane. In short: state on-chain, computation off-chain, encrypted bytes in storage.



three planes and their main components

The client and the services sit in the off-chain plane. The drawing shows ProofService in the services band, but it is a benchmark and seed helper, not a process the demo starts (D23). The client also sends contract calls straight from the wallet, and the services call the chain too. Only contract calls, proofs, and events cross to the chain, and only encrypted bytes reach storage.

Plane	Runs on	Holds	Never holds
On-chain	Hardhat local network, Solidity	Identities and public keys, roles, consent tokens, record references (CID and hash), claim commitments, share commitments, break-glass and Tier 2 state, audit events	Plaintext records, raw shares, salts, private keys
Off-chain (client and services)	React apps in the browser, plus Node.js processes	All computation: FHIR, AES-GCM, ML-DSA, ML-KEM, proofs, Shamir. The root secret S sits in a passphrase-encrypted device keystore. Derived keys live in tab memory only	Any persistent plaintext secret. Nothing reaches a service unless it is wrapped to a one-time key. Record crypto runs in the client library, never in a service that receives keys *(v1.2, D12)*
Storage and custody	IPFS through Pinata, plus trustee devices	Encrypted record envelopes. Encrypted share bundles, one per trustee	Anything readable without a key held somewhere else

Component catalog and dependencies

Twenty components make up the system; two are new in v1.2.

Component	Plane	Responsibility	Owner
PatientIdentity.sol	On-chain	Wallet and DID link, public-key registry (ML-DSA and ML-KEM keys). The DID hash is bound to the registering address. Clinicians, labs, and Hospital B register the same way	Anush
PatientAccount.sol	On-chain	ERC-4337-style account: the patient signs an EIP-712 message and a relayer pays gas	Anush
PatientAccountFactory.sol *(new, D10)*	On-chain	Deploys one account per owner wallet at a precomputable address and resolves wallet to account	Anush
RoleRegistry.sol	On-chain	ABAC roles: Doctor, Lab, Insurer, IoTDevice, EmergencyResponder	Anush
ConsentRegistry.sol	On-chain	Consent tokens, wrapped-key delivery through `KeysWrapped`, access checks (`canAccess` free, `checkAccess` logged), proof hook	Anush
RecordRegistry.sol	On-chain	Record references (CID and hash) by patient and resource type	Anush
ProofVerifier.sol and ProofGateway.sol	On-chain	Groth16 verification, claim commitments, nonce replay protection, batching	Punith

Component	Plane	Responsibility	Owner
AuditLog.sol	On-chain	One append-only event feed. Only allow-listed contracts may write to it	Punith
RecoveryVault.sol	On-chain	Share commitments, k-of-(n+1) rules with the trustee-mix checks, Tier 2 co-sign and delay, lost-key requests, source of truth for request state	Rohith, Yagna
BreakGlassCapsule.sol	On-chain	Emergency role check, capsule pre-authorization, time-boxed window	Yagna
Client data library (was DataService) *(v1.2, D12)*	Client and Node scripts	FHIR build, AES-GCM, ML-DSA sign and verify, ML-KEM wrapping, registry calls. Runs where the keys are, never as a service that receives them	Rohith
IPFS proxy *(new, D12)*	Off-chain, Node.js, 127.0.0.1:4101	Holds the Pinata credentials, pins and fetches ciphertext envelopes, keeps a local cache	Rohith
ProofService	Off-chain, Node.js	A benchmark and seed helper for witness and Groth16 proof generation and batching. Not one of the demo's processes, because the browser generates the demo's proofs (D23)	Punith
RecoveryDealer	Off-chain, one-shot CLI	Setup-phase dealing only. Never a running service	Rohith

Component	Plane	Responsibility	Owner
RecoveryCoordinator	Off-chain, Node.js	Share collection, commitment check, Lagrange reconstruction, audit-record signing. Driven by a scripted guardian harness in the demo	Rohith, Yagna
Relayer	Off-chain, Node.js	Submits patient-signed actions, creates accounts through the factory, and pays gas	Anush
Seed script	Off-chain, Node.js	Demo identities on Hardhat's unlocked accounts, synthetic data, the demo claim, and the git-ignored demo fixture	Rohith
Device keystore	Client	Passphrase-encrypted store for one secret per party	Rohith, Yagna
Frontend SPA	Client, React	Patient dashboard, records, responder and clinician views	Yagna
Hospital B viewer	Client, React	Second small app on its own port that consumes the same tokens	Rohith

How the contracts depend on each other

Every state change that matters ends in an `AuditLog` call. If that call reverts, the whole transaction reverts, so nothing succeeds without being logged. Rows marked v1.2 add edges that v1.1 missed.

Contract	Reads or calls	Called by
PatientAccountFactory *(v1.2)*	Deploys PatientAccount	The relayer, the deploy script, anyone

Contract	Reads or calls	Called by
PatientAccount	Any target, on the patient's behalf	The relayer (`executeSigned`), the owner wallet (`execute`)
PatientIdentity	Nothing	RecordRegistry, RecoveryVault, ConsentRegistry, ProofGateway *(v1.2)*, BreakGlassCapsule *(v1.2)*, and every client that verifies a signature or wraps a key
RoleRegistry	Nothing	ConsentRegistry, ProofGateway, BreakGlassCapsule
ConsentRegistry	PatientIdentity, RoleRegistry, AuditLog, ProofGateway (the proof hook set by `setProofVerifier`)	Patients, grantees, ProofGateway (token reads)
RecordRegistry	PatientIdentity, AuditLog	The patient's account, every reader
ProofGateway	ProofVerifier, ConsentRegistry, RoleRegistry, PatientIdentity *(v1.2)*, AuditLog	Issuers, verifiers, the patient
RecoveryVault	PatientIdentity, AuditLog	BreakGlassCapsule (`setReleaseMode`), the deploy script (`setBreakGlassCapsule` , once), the coordinator (reads), patients, guardians, the custodian
BreakGlassCapsule	RoleRegistry, PatientIdentity *(v1.2)*, AuditLog, RecoveryVault (`setReleaseMode` and request-state reads)	Emergency responders, anyone for `closeWindow`
AuditLog	Nothing	Every allow-listed logging contract

What each part can see

Nine parties each see something different, and none sees plaintext except the holder of the right key.

Party	Can see	Cannot see
Anyone reading the chain	Record types, counts, timestamps, creators, CIDs, consent terms, who accessed what and when, commitments. It does see wrapped type keys in `KeysWrapped` events, which only the named grantee can open	Plaintext, raw shares, salts, secret keys
IPFS and Pinata	Encrypted envelopes	Anything readable
The IPFS proxy *(v1.2)*	Encrypted envelopes in transit and in its local cache	Any key or plaintext. It rejects anything that is not a well-formed envelope
A grantee holding a token	Records of the granted types, after unwrapping the type key	Other types, the capsule key, S
The relayer	Signed actions to submit	It holds no key, so it can delay or drop an action but never forge one
The coordinator	Shares and S in memory, briefly, then it wipes them	In break-glass mode it decrypts the emergency capsule only, never any other patient record, and it never persists anything (D20)
The dealer	S for the length of dealing, then it overwrites what it can and exits	Anything afterwards
One trustee	Its own encrypted bundle, which holds one share	Other trustees' shares
Any k colluding trustees	Can rebuild S, so they can read everything and forge the patient's signatures	Mixed trust categories, with the category cap, are the mitigation

Cross-cutting design

Seven design rules hold across every mechanism below.

- **One root secret.** S is created on the patient's device. The signing key pair, the encryption key pair, and every record, type, and capsule key derive from it with HKDF-SHA256. Rebuilding S restores the whole identity. The Identity, keys, and accounts section has the tree.
- **Post-quantum scope.** Record and event signatures use ML-DSA-65, and key and share transport use ML-KEM-768, both behind one `PQCProvider` interface. The Groth16 proofs and Ethereum transaction signatures stay classical.
- **Policy before cryptography.** The on-chain role and capsule checks run first. A failure reverts the call, so no cryptography runs and no state changes.
- **Verification off-chain, commitments on-chain.** A raw share or salt is never calldata. The coordinator checks shares against on-chain commitments in memory.
- **One audit feed.** `AuditLog` is the canonical on-chain trail. What the chain cannot see, such as which trustees responded, goes into signed off-chain audit records, stored as one append-only JSONL file per patient and signed by the coordinator's registered demo identity.
- **One clock** *(v1.2, D11)*. Contracts use `block.timestamp`. The demo node mines a block every second, so block time follows wall-clock time. UI countdowns and the coordinator's deadlines (`COLLECTION_TIMEOUT`, and `FULL_MODE_TTL` for the lost-key path, which has no on-chain window) all read block time. The demo never advances the clock. Tests that need to run on a separate in-process network. v1.1 said countdowns "can be advanced in tests", which would have moved chain time away from the coordinator's clock if done on the demo node.
- **Randomness.** CSPRNG only, never `Math.random`, wherever a secret, salt, or key is made.

Deployment topology

Six local processes run on one machine, started by one command, plus one external dependency.

Process	Where	Notes
Hardhat node	127.0.0.1:8545, chain ID 31337	The only chain. solc 0.8.24, `evmVersion` cancun, Hardhat 2.x pinned. Interval mining at one block a second for the demo *(v1.2, D11)*
Frontend SPA (Vite)	Port 5173	Patient, clinician, and responder screens. React 18, Vite, ethers.js 6, injected wallet
Hospital B viewer	Port 5174	Second app on the same contracts. Calls only the IPFS proxy's read-only GET route (D18)
Coordinator API and dealer launcher	127.0.0.1:4100	One local service, loopback only, bearer tokens, CORS limited to the frontend origin *(v1.2, D17)*
IPFS proxy	127.0.0.1:4101	Holds `PINATA_JWT` and `PINATA_GATEWAY`. Sees ciphertext only *(v1.2, D12)*
Relayer	Local Node process	Funded local account that pays gas and creates patient accounts
IPFS through Pinata	Cloud	The only external dependency. The proxy's local cache is the demo's offline fallback (PRD risk table)

Repo layout: `/contracts`, `/circuits`, `/services` (data library, proof, recovery, IPFS proxy), `/frontend`, `/test`, `/scripts`, `/fixtures`. One command, `npm run demo:breakglass`, runs the node, deploy, seed, services, and the scripted trigger.

The window and Tier 2 timers are contract timers and follow chain time. `COLLECTION_TIMEOUT` and `FULL_MODE_TTL` are coordinator-side deadlines, and since v1.2 they also read chain time, so all four agree.

Deploy wiring

`scripts/deploy.js` performs these steps. The TRD states the register, wire, and assert steps and the errata state the capsule setter. The deployment order itself follows from the dependencies and is inferred.

1. Deploy `RoleRegistry`, `PatientIdentity`, `PatientAccountFactory` *(v1.2)*, and `AuditLog`, which depend on nothing.
2. Deploy `ConsentRegistry`, `RecordRegistry`, the generated `ProofVerifier`, `ProofGateway`, then `RecoveryVault`, then `BreakGlassCapsule`. The capsule is deployed after the vault, because its constructor takes the vault address (errata 16).
3. Call `RecoveryVault.setBreakGlassCapsule` once, now that both exist (errata 16).
4. Register every logging contract with `AuditLog.registerEmitter`, including RecordRegistry and ProofGateway.
5. Call `ConsentRegistry.setProofVerifier` with the ProofGateway address (inferred from the TRD's wire addresses).
6. Assert `WINDOW_DURATION` is at least twice `TIER2_DELAY`, and refuse to deploy otherwise.
7. Write the deployed addresses to config.

The TRD does not fix constructor arguments. The order above follows from the dependencies.

Timing parameter	Demo value	Where
`WINDOW_DURATION`	180 s	BreakGlassCapsule constructor
`TIER2_DELAY`	60 s	RecoveryVault constructor
`COLLECTION_TIMEOUT`	20 s	Coordinator config, off-chain
`FULL_MODE_TTL`	300 s	Coordinator config, off-chain, lost-key requests

Demo run order

`npm run demo:breakglass` orchestrates the node start, deploy, seed, services, and the scripted break-glass sequence (BG-8, E2E-1):

1. Start the Hardhat node with interval mining.
2. Run `deploy.js`: deploy, wire, and write addresses.
3. Run `seed-synthea-data.js`: load the Synthea FHIR data, create the demo identities on Hardhat's unlocked accounts, register their public keys, have the scripted patient share a Claim permission with Dr. Rao, have Dr. Rao issue the demo claim, and write the demo-only fixture.

4. Start the coordinator and dealer launcher (4100), the IPFS proxy (4101), the frontend (5173), and the Hospital B viewer (5174).
5. Run the scripted break-glass trigger.

The other key flows (routine sharing, lost-key recovery, cross-hospital access) are not part of this one command.

Key architecture decisions and limits

Twenty-two decisions shape the design, and ten limits follow from them.

Decision	Choice	Why
Where shares are checked	Off-chain, against on-chain commitments	A share sent as calldata would be public forever
Break-glass release (D1)	Capsule fields only. A full record is never assembled on this path	Limits what an emergency exposes
Threshold rule (D2)	k-of-(n+1) with k at least 2, enforced in the contract	A malformed direct call still fails
Share verification	Hash commitments, not Feldman VSS	Stays post-quantum consistent. The patient's own device is the dealer
Guardians in the demo (D5)	Scripted. One configuration: four guardians plus the hospital backup, three of the four needed	Makes the demo repeatable and the screens consistent
Patient account (D7)	ERC-4337-style, no bundler or paymaster	Hides gas without the full standard
Record signer (D9)	The creator signs with ML-DSA, and the registry stores the creator	Readers can verify who wrote a record
Request state	RecoveryVault is the source of truth. BreakGlassCapsule keeps only the window (errata 2)	One owner for patient, requester, mode, cancelled, and Tier 2
Dealer	One-shot CLI that overwrites what it can and exits, never a service	Shrinks the time S exists outside the keystore

Decision	Choice	Why
PQC library	Behind a `PQCProvider` interface	A library swap never touches protocol logic
Account creation (D10)	A `PatientAccountFactory` creates accounts and resolves wallet to account	The first screen could not otherwise find a patient's identity
One clock (D11)	Interval mining, deadlines read block time, no clock jumps in the demo	Countdowns cannot freeze or disagree with the coordinator
Record crypto (D12)	A client library does the crypto. A loopback IPFS proxy holds credentials and sees only ciphertext	No key reaches a service, and no credential reaches the browser
Access checks (D13)	A free `canAccess` view and a logged `checkAccess` transaction	A client cannot read the return value of a logging call
Trustee mix (D14)	No category may hold k or more of the n+1 trustees, plus the existing all-one-category revert	k trustees of one kind could otherwise rebuild S
Local-service security (D17)	Session token, origin check, and a fetch-based stream reader	`EventSource` cannot send a token
Hospital B read route (D18)	The IPFS proxy's GET route accepts the Hospital B origin. Pinning stays frontend-only	Hospital B must fetch ciphertext, and the proxy's cache serves demo beat 11 offline
Proof request and result (D19)	A request shape, and an EligibilityVerified event from ProofGateway	The verifying call changes state, so its return value is unreadable to a client
Capsule-mode authorization (D20)	The requester signs a challenge, the coordinator checks the on-chain mode and window, and the capsule is registered before it is authorized	Anyone could otherwise open a request, and a half-saved capsule could be released

Decision	Choice	Why
Tier 2 guards (D21)	custodianCoSign reverts for an unknown, cancelled, completed, or already co-signed request. Limit 20 says the delay is not cryptographic	A second call could reset the delay
Local-service tokens (D22)	A session token starts requests. A per-request token guards events, result, close, and shares	Separates who may start a request from who may read it
Small spec fixes (D23)	8-bit range checks, ProofService as a helper, and a late k-th share rebuilding at once	Closes three loose ends

Limits that shape the architecture

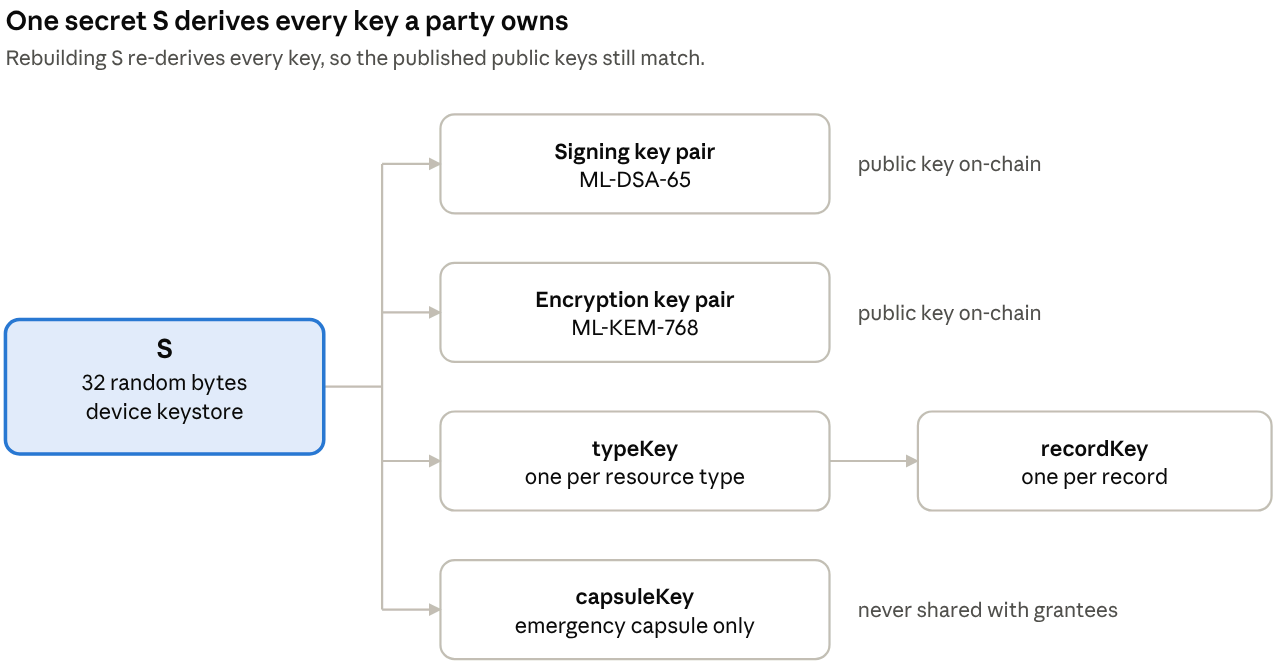
The TRD lists 19 known limits. These come from the architecture itself:

1. Any k colluding trustees can rebuild S and forge signatures. Keys cannot rotate without a new S.
2. Capsule-only release is enforced by the flow, not by separate keys, because the emergency quorum rebuilds S.
3. A revoked grantee keeps type keys already delivered, and CIDs are public. Consent is time-limited by the contracts, not by the keys.
4. Record metadata on the chain is public.
5. The coordinator briefly holds S in both modes, and overwriting memory is best effort in JavaScript.
6. The relayer is trusted for liveness only.
7. Tier 2 helps only when exactly one guardian share is missing, because the custodian holds a single share. With k = 2, the custodian plus one guardian can rebuild S.
8. Post-quantum protection covers signatures and key transport, not the proof system.
9. There is no out-of-band notification, so an unconscious patient cannot cancel a Tier 2 request *(new in v1.2)*.
10. The Tier 2 delay is not cryptographic: the custodian can decrypt its own bundle at any time, so the delay holds only because the contract records when it ends and the coordinator honors it *(new in the second pass)*.

Identity, keys, and accounts

One secret, S, per party creates every key that party owns, and rebuilding S restores the whole identity. This section covers the key hierarchy, the algorithms, vault creation, the device keystore, and how a patient acts on-chain without handling gas.

Key hierarchy



key hierarchy · one root, five derived keys

A grantee holding one `typeKey` can read records of that type, including later ones. It cannot derive S, another type's key, or the `capsuleKey`. Clinicians, labs, Hospital B, and the demo Coordinator derive their own two key pairs the same way from their own secret and register the public keys. They hold no type keys of their own, because type keys reach them wrapped.

Key	Derivation	Used for	Who may hold it
S	32 bytes from a CSPRNG at vault creation	Root secret, split into n+1 shares for recovery	Patient's keystore. Briefly the dealer and the coordinator. Rebuildable by any k trustees

Key	Derivation	Used for	Who may hold it
Signing key pair	`ml_dsa65.keygen(HKDF(S, 'medichain-dsa-v1', 32))`	Records and signed audit records	Patient (private). Public key in PatientIdentity
Encryption key pair	`ml_kem768.keygen(HKDF(S, 'medichain-kem-v1', 64))`	Receiving wrapped type keys and shares	Patient (private). Public key in PatientIdentity
typeKey	`HKDF(S, 'medichain-type-v1:' + resource type name)`	One per type, wrapped to a grantee at consent time	Patient. Each grantee holds the types it was given
recordKey	`HKDF(typeKey, 'medichain-record-v1:' + recordId)`	AES-GCM key for one record	Derived on demand by anyone holding the typeKey
capsuleKey	`HKDF(S, 'medichain-capsule-v1')`	Encrypts the emergency capsule only	Patient. The coordinator derives it in break-glass. Never shared with grantees

Algorithms and parameters

Purpose	Algorithm	Setting	Library
Record encryption	AES-256-GCM	96-bit random IV, 128-bit tag, a new IV for every encryption	`node:crypto`
Signatures	ML-DSA-65 (FIPS 204)	Public key 1,952 B, signature 3,309 B	`@noble/post-quantum`
Key encapsulation	ML-KEM-768 (FIPS 203)	Public key 1,184 B, ciphertext 1,088 B	`@noble/post-quantum`
Key derivation	HKDF-SHA256	32-byte keys	`node:crypto hkdfSync`
Secret sharing	Shamir, k-of-(n+1), 2 <= k <= n+1	GF(2^8). Shares are odd-length hex strings	secrets.js-grempe

Purpose	Algorithm	Setting	Library
Share commitment	SHA-256	Over `utf8(shareString)`, the salt, and `utf8('medichain-share-v1')`, concatenated. Salt at least 16 B	`node:crypto`
Keystore	PBKDF2-HMAC-SHA256, then AES-256-GCM	At least 600,000 iterations, 16 B salt, 12 B IV	Web Crypto or `node:crypto`
Zero-knowledge	Groth16 on BN254	Proof is 8 field elements. Poseidon commitment	circom, snarkjs, circomlibjs
Account actions	EIP-712, secp256k1 owner key	Nonce and deadline bound	ethers, OpenZeppelin
Randomness	CSPRNG only	`crypto.randomBytes` or `crypto.getRandomValues`, never `Math.random`	Platform

`@noble/post-quantum` is pre-1.0 (0.7.1 tested, including in the v1.2 review), so pin the exact version, use `.js` subpath imports, and Node 20.19 or later. Every call goes through the `PQCProvider` interface, so another backend can replace it.

Rules that matter

- **Share encoding.** A share is the exact string `secrets.share()` returns, and the commitment covers its UTF-8 bytes. Never hex-decode it: shares are odd-length hex (99 characters for a 32-byte secret), and `Buffer.from(share, 'hex')` silently drops the last nibble (errata 4).
- **S is wrapped only twice.** To the dealer's one-time key when splitting or re-sharing, and to the client's one-time key in lost-key recovery. It never leaves a process in plaintext.
- **Sharing is by type.** A key already delivered cannot be recalled.
- **No rotation without a new S.** Rotating means re-encrypting every record and re-dealing the shares. `updateKeys` is not used.
- **Policy before cryptography** on every emergency path.

Vault creation (SCR-02) *(v1.2)*

v1.1 started at the registration call, with no step that created the patient's account.

1. If the wallet has no account, the relayer calls `PatientAccountFactory.createAccount(wallet)` and pays the gas (D10). The app resolves the account with `accountOfOwner`.
2. The client creates S with `crypto.getRandomValues` and derives the signing and ML-KEM key pairs.
3. Through the patient's account it calls `PatientIdentity.register(didHash, dilithiumPubKey, mlkemPubKey)`. `patientId` is the DID hash, the `keccak256` of the string `did:medichain:` plus the account's lowercase address, and the contract checks that binding (PAT-8, D15).
4. It writes S to the device keystore.

Device keystore

One encrypted record per party, holding one secret: S for a patient, the party's own secret for everyone else. The browser keeps it in IndexedDB and can export a backup file. A wrong passphrase fails the GCM tag, so there is no separate password check. Unlocked key pairs live in React state for the open tab only, and are never written in plaintext, logged, or sent to a service except wrapped to a one-time key. A lost passphrase with no backup goes through guardian recovery.

Accounts and the relayer

1. The patient signs an EIP-712 `Execute(target, data, nonce, deadline)` message with the owner key.
2. The relayer calls `PatientAccount.executeSigned(target, data, deadline, sig)` and pays gas. This is simulated sponsorship, with no bundler or paymaster.
3. The account checks the signature and deadline, increments its nonce before the call, then calls the target.
4. The target sees the account's address as `msg.sender`, so every contract treats the account as the patient.

The relayer can delay or drop an action but cannot forge one, and replay or late submission both revert. If the sponsored path is dropped, the app falls back to direct wallet transactions.

What the derivation buys and costs

In return for one secret, lost-key recovery restores the whole identity: the same public keys re-derive, old signatures still verify, and old wrapped keys still unwrap. The costs:

- Any k colluding trustees can rebuild S, so they can read everything and forge the patient's signatures.
- Keys cannot rotate without a new S.
- A revoked grantee keeps a working key for later records of the same types.
- Deriving `capsuleKey` from S does not separate emergency access from full access, because anyone who rebuilds S can derive both. A real separation needs an independently shared capsule secret and its own dealing run, which is out of scope.
- Wallet recovery is out of scope. Post-quantum protection covers signatures and key transport, not the owner's secp256k1 signature.

Data layer: record write and read

Every health record is encrypted, signed, and pinned to IPFS before anything about it reaches the chain. The chain stores only the CID, a hash, and the creator, and an authorized reader verifies and decrypts without trusting IPFS.

Write path (patient's client with the client data library)

1. Build the FHIR R4-inspired resource (Patient or Observation). It carries `resourceType`, `id`, and a `subject` reference. A Patient uses its `id` instead (errata 13). Consent is a read-only projection and is never written *(v1.2: v1.1 listed it here)*. Ids follow the FHIR id pattern, and DIDs travel as identifiers *(D16)*.
2. Derive `recordKey = HKDF(typeKey, 'medichain-record-v1:' + recordId)`. The `recordId` is the FHIR id, known before upload.
3. Encrypt with AES-256-GCM, using a fresh 12-byte IV and a 16-byte tag.
4. Sign the ciphertext with the creator's ML-DSA-65 key (DAT-4, decision D9).
5. Build the `RecordEnvelope` and send it to the IPFS proxy, which pins it through Pinata and returns the CID *(v1.2, D12)*.
6. Through the patient's account, call `RecordRegistry.addRecord(patientId, resourceType, cid, contentHash, creator)`. `contentHash` is the sha256 of the ciphertext. The contract logs RecordAdded.

Read path (authorized retrieval, DAT-3)

1. The grantee's client calls the free `ConsentRegistry.canAccess(tokenId, requester, resourceType)` to decide whether to continue. To leave the audit trail it then sends `checkAccess` as a transaction from a funded signer and reads AccessGranted or AccessDenied from the receipt. A denial stops here and is logged *(v1.2, D13)*.
2. `RecordRegistry.recordsOf(patientId, resourceType)` lists the records. The registry is the only place that knows the CIDs.
3. Fetch each envelope through the IPFS proxy by CID and check `sha256(ciphertext)` against `contentHash` before anything else.
4. Look up the creator's key with `PatientIdentity.getDilithiumKey` and verify the signature.
5. Read the `KeysWrapped` event, unwrap the `typeKey` with the reader's own ML-KEM secret key, derive the `recordKey`, and decrypt. A failed GCM tag aborts with no partial output.
6. Return the FHIR JSON.

The same read path serves the clinician's Shared with me screen and the Hospital B viewer, which uses its own demo identity and the same token unchanged (INT-1).

What each store holds

The envelope carries `schemaVersion`, `resourceType` (the name, not the hash), `recordId`, `ciphertext`, `iv`, `authTag`, and a `signature` (alg, signerId, value). A reader checks them in this order: hash, signature, then decrypt. Only the patient's account may call `addRecord`. The `cid` is a string, because a 32-byte field cannot hold a CIDv1. Field lists are in the Backend Schema.

Store	Holds	Never holds
IPFS via Pinata, through the proxy	The `RecordEnvelope`	Keys or plaintext
RecordRegistry	`entryId`, `patientId`, `resourceType`, `cid`, `contentHash`, `creator`, `createdAt`	Anything encrypted or readable
ConsentRegistry events	Wrapped type keys, as ciphertext	Plaintext keys
Device keystore	S	Records

Records written by someone else

In the prototype a clinician or lab cannot register a record directly, so the seed script simulates records authored by others: it encrypts under the patient's `recordKey`, signs with the creator's key, and registers with the creator set. A production route would have the creator encrypt to the patient's ML-KEM key and sign a hash of the plaintext inside the payload, and the patient's client would re-encrypt under the record key and register it (known limit 18).

The emergency capsule

The capsule is stored the same way, with two differences. It is encrypted under `capsuleKey`, which is never shared, and it registers under its own `EmergencyCapsule` type. A consent token can never grant that type. The client library produces it from the `CapsulePayload` (errata 7).

Limits

- Record metadata on the chain is public: types, counts, timestamps, creators, and CIDs.
- A revoked grantee keeps type keys already delivered, and CIDs are public.
- The signature covers the ciphertext only, a documented limitation (errata 14).
- Pinata plan limits on file count and size must be confirmed before the demo.
- Evaluation uses Synthea data in FHIR R4 form, so results may not reflect real record shapes.

Consent and access control

A consent token says who may read which resource types until when, and a wrapped type key delivers the means to decrypt. Every use is checked against the token and the grantee's role, and the outcome is logged (CON-1 to CON-4, DAT-5). Keys travel in an event, and expiry and revocation make a token invalid, but keys already delivered stay with the grantee.

Issuing a consent

1. The patient picks the grantee, a role, the resource types, and an expiry in the future.
2. For each type the client derives the `typeKey` from S and encapsulates it to the grantee's ML-KEM key (`PatientIdentity.getMLKEMKey`). Each result is a `WrappedTypeKey` , serialized as UTF-8 JSON (errata 5).
3. The patient signs an EIP-712 Execute message. The relayer calls `ConsentRegistry.issueConsent(grantee, roleId, resourceTypes, expiresAt, wrappedTypeKeys)` .
4. The contract checks that the caller is a registered patient account, the expiry is in the future, and there is one wrapped key per type. It rejects the EmergencyCapsule type. A Claim-type entry is empty.
5. It stores the token under `tokenId = keccak256(abi.encode(block.chainid, address(this), msg.sender, tokenCounter))` (errata 6), emits `KeysWrapped` (the keys are emitted, never stored), and logs ConsentIssued.

Checking access *(v1.2, D13)*

There are two calls. `canAccess(tokenId, requester, resourceType)` is a free view call that screens use to decide what to show. `checkAccess` is a transaction with the same logic that requires `msg.sender == requester` , emits AccessGranted or AccessDenied, and never reverts on a denial, so the denial is logged. A client that needs the result sends the transaction from a funded signer and reads the event from the receipt. v1.1 had one function that both logged and returned a value, which a client could not read without sending a transaction.

Both return true only if the token is not revoked, not expired, includes the resource type, and the requester's role matches. `roleId` is `keccak256(bytes(roleName))` , and the contract resolves it to the `Role` enum by name (errata 3). After a true result the grantee follows the read path in the data layer section.

Roles (ABAC)

The `Role` enum has Doctor, Lab, Insurer, IoTDevice, and EmergencyResponder, plus None. An admin assigns roles through RoleRegistry, which builds on OpenZeppelin `AccessControl`. Doctor and Lab can register claim commitments when they hold a Claim token. EmergencyResponder is what triggers break-glass.

Revocation, expiry, and claim tokens

- `revokeConsent(tokenId)` works only for the issuing patient. It is one storage update plus an audit event. Expiry needs no action, because `isValid` compares against the stored `expiresAt`.
- Neither recalls keys already delivered. A revoked grantee can still derive keys for later records of the same types, and the CIDs are public (known limit 6). Consent is time-limited by the contracts, not by the keys.
- A token for the Claim type carries an empty wrapped-key entry, because the issuer reads nothing. It lets a Doctor or Lab register a claim commitment (see Zero-knowledge claims).

Second institution: Hospital B (INT-1, INT-3)

Hospital B is a second small app on its own port, with a demo identity from the seed fixture. It calls the same contracts and the same retrieval path: it reads `tokenIdsOfGrantee`, reads `KeysWrapped`, then fetches and decrypts. There is no copy of the data and no extra trust. The same token works on both screens, which is what shows portability across institutions.

Where the checks run

Check	Where	Why there
Registered patient, future expiry, key count	Inside `issueConsent`	A malformed direct call still fails
Token validity and role	Inside `canAccess` and `checkAccess`	Logged either way when sent as a transaction
Hash, signature, and GCM tag	The reader's client	Nothing from storage is trusted

Check	Where	Why there
Key possession	The grantee's own ML-KEM secret	Only the named grantee can unwrap

Limits

- Token metadata is public on the chain: who, which types, and until when.
- Revocation cannot recall delivered keys, and a grantee holding a type key can decrypt later records of that type.
- Doctor- and lab-created records still go through the patient's client (known limit 18).

Tier 1 recovery

Tier 1 lets a patient who loses their device restore the root secret S from trustees, and it supplies the guardian quorum that break-glass uses. S is split once, in a single dealing run, into n+1 shares with threshold k, where 2 <= k <= n+1 is enforced on-chain (T1-2, decision D2). There are n guardian shares (family, hospital custodian, neutral guardian service) and one for the institutional custodian, who is trustee n+1. The demo uses n = 4 and k = 3. Tier 2 is part of the same scheme, not a separate mechanism: the custodian holds one share, so Tier 2 lets recovery succeed with k-1 guardians plus the custodian, and it can never act alone.

Design rule: commitments are the only thing that touches the chain. Shares and salts are checked and combined off-chain, because a raw share sent as calldata would be public forever. The three phases below are setup, custody, and recovery.

Setup (once, with a one-shot dealer)

1. The patient picks n guardians, one institutional custodian, and a threshold k. The UI recommends k no greater than n, and the contract allows up to n+1.
2. The UI warns if every guardian is in the same trust category, or if any category would hold k or more trustees, and the contract rejects both (errata 12, D14). The custodian row is excluded from the first check.
3. The client calls `POST /dealer/run` on the local launcher with n, k, and each trustee's index, category, and ML-KEM public key. The launcher starts a one-shot dealer, which makes a one-time ML-KEM key pair and returns its public key.

4. The client wraps S to that key and calls `POST /dealer/:sessionId/secret`. S never travels in plaintext, even on loopback.
5. The dealer splits S with secrets.js-grempe, using a CSPRNG for the polynomial and the salts. For each share it computes `SHA256(utf8(shareString) || salt || utf8('medichain-share-v1'))`. `shareString` is the exact string `secrets.share()` returns, never hex-decoded (errata 4). The salt travels inside the encrypted bundle and never on-chain.
6. It encrypts each share and salt to its trustee's ML-KEM public key and returns the n+1 bundles and commitments. Then it overwrites what it can and exits (best effort in JavaScript).
7. The client adds each trustee's address and category and calls `RecoveryVault.registerShareCommitments(commitments, k)` through the patient's account. Only commitments reach the chain.
8. The client hands each bundle to its trustee.

Custody (steady state)

Each trustee holds one encrypted bundle, which holds one share. The chain holds commitments only. Trustees span mixed trust categories, plus the institutional custodian row, so one compromised kind of relationship cannot approach the threshold. The vault also requires unique, contiguous indexes 1 to n+1, exactly one `InstitutionalCustodian` row at index n+1, and unique trustee addresses.

Recovery (emergency path)

1. **Policy first.** The role and capsule check (break-glass) or the patient-account check (lost key) runs before any cryptography.
2. The coordinator reads `RecoveryVault.getCommitments(patientId)`, a free view call.
3. Each responding trustee decrypts its own bundle, then re-wraps the share and salt to the coordinator's one-time ML-KEM key for that single hop. A raw share never travels unencrypted, even internally.
4. The coordinator checks every share against its commitment. A tampered share is rejected before reconstruction, not after.
5. With k valid shares it runs Lagrange interpolation off-chain and recovers S.
6. What happens next depends on the release mode: capsule mode (see Break-glass and Tier 2) or full mode (next).

7. The event is logged on-chain, and the coordinator writes a signed off-chain audit record that lists which trustees responded.

Lost-key path (full mode)

The patient's wallet is assumed intact. Recovery restores access to records and the identity keys, not control of a lost wallet.

1. On a new device the client makes a one-time ML-KEM key pair, and the patient's account calls `RecoveryVault.startRecovery(patientId, clientKeyHash)`, where `clientKeyHash` is the sha256 of that public key. This opens a FullRecord request and logs RecoveryInitiated.
2. The client calls `POST /requests` with mode full, the request id, and the one-time public key.
3. The coordinator reads `getRecovery` and refuses unless the request is FullRecord, was opened by the patient's account, is neither cancelled nor completed, and the key's hash matches the stored `clientKeyHash`.
4. Steps 2 to 5 of the recovery path run unchanged, including the Tier 2 fallback and its cancel right.
5. The coordinator wraps S to the client's key, serves it once through `GET /requests/:id/result`, and wipes S. It never reads a record or ciphertext, because the client decrypts locally.
6. The client unwraps S, asks for a new passphrase, writes a new keystore, and re-derives its keys. They match the published ones, so old signatures verify and old wrapped keys unwrap.
7. The client calls `completeRecovery(requestId)`, which requires FullRecord mode (errata 17) and logs RecoveryCompleted. The client then offers to refresh the guardians' shares.
8. A full-mode request has no on-chain window, so the coordinator enforces `FULL_MODE_TTL`, 300 s in the demo, wipes its keys then, and mirrors that deadline in the event's `windowExpiry` (errata 18). All deadlines read chain time (D11).

Proactive re-sharing (T1-6)

1. The patient starts it from their own client, prompted by a Refresh due reminder. A fresh one-shot dealer receives S from the keystore, wrapped as in setup, builds a new random polynomial, and produces new shares, salts, and commitments.

2. Each new bundle is encrypted to its trustee's ML-KEM key, and the dealer overwrites what it can and exits.
3. The patient calls `RecoveryVault.rotateShareCommitments(fresh, k)`. It stores the new commitments, sets `updatedAt`, and increments the epoch. A share from an earlier epoch no longer matches any current commitment, so the coordinator rejects it.

Cadence is a team policy that the contract does not enforce, and nothing schedules a refresh. `getGuardianSet(patientId)` returns threshold, epoch, and updatedAt, so the app can show Refresh due (errata 10). An attacker who already holds k shares of one generation can still rebuild S offline.

Service interfaces

Interface	Detail
Dealer launcher	`POST /dealer/run` and `POST /dealer/:sessionId/secret`. Loopback only, with a per-session bearer token generated at startup
Coordinator API	`POST /requests`, `POST /requests/:id/shares`, `GET /requests/:id/events` (SSE, read by a fetch stream with the token in a header), `GET /requests/:id/result` (409 before release, 410 after close), `POST /requests/:id/close`
Progress event	`requestId`, `phase`, `needed`, `validCount`, `trustees` (index, category, status), `windowExpiry`, `tier2EndsAt`. Phase is one of collecting, tier2-wait, reconstructing, released, failed, cancelled, closed
Rules	Reject shares not wrapped to the one-time key. Never release after `windowExpiry`. Wipe the key and any released data at close. Check policy before any cryptographic call

Where each check runs

Check	Where	Why there
2 <= k <= n+1, mixed categories, category cap	RecoveryVault, on-chain	A malformed direct call still fails
Share against its commitment	Coordinator, off-chain, in memory	A share must never be calldata
Request is FullRecord, opened by the patient, key hash matches	Coordinator, after reading `getRecovery`	Binds the one-time key to the patient
Mode on completion	`RecoveryVault.completeRecovery`	A break-glass responder cannot log a completion

Limits

- secrets.js-grempe is unmaintained, with no release since September 2019. It passed an end-to-end test of this flow.
- Any k colluding trustees can rebuild S.
- Hash commitments detect tampering, not inconsistent dealing. The patient's own device is the dealer, which is why this is acceptable.
- The dealer and the coordinator each hold S briefly, and overwriting memory is best effort.
- Tier 2 only helps when exactly one guardian share is missing (see Break-glass and Tier 2).

Break-glass and Tier 2

Break-glass lets a verified emergency responder see a small, pre-authorized summary when the patient cannot consent. It is time-boxed and fully audited, and the policy check runs before any cryptography. Tier 2 is a delayed fallback with a cancel right, for when exactly one guardian share is missing. This is the centerpiece demo feature.

What it guarantees

- Time-limited access to capsule fields only, never the full record (BG-1 to BG-5, decision D1).
- The patient pre-authorizes the fields in advance (BG-4).
- One window covers the whole request, including the Tier 2 wait (BG-8).
- A delayed custodian fallback that the patient or any guardian can cancel (BG-6, BG-7).
- Every step is logged, so a request cannot succeed without a trace (AUD-1, AUD-2).

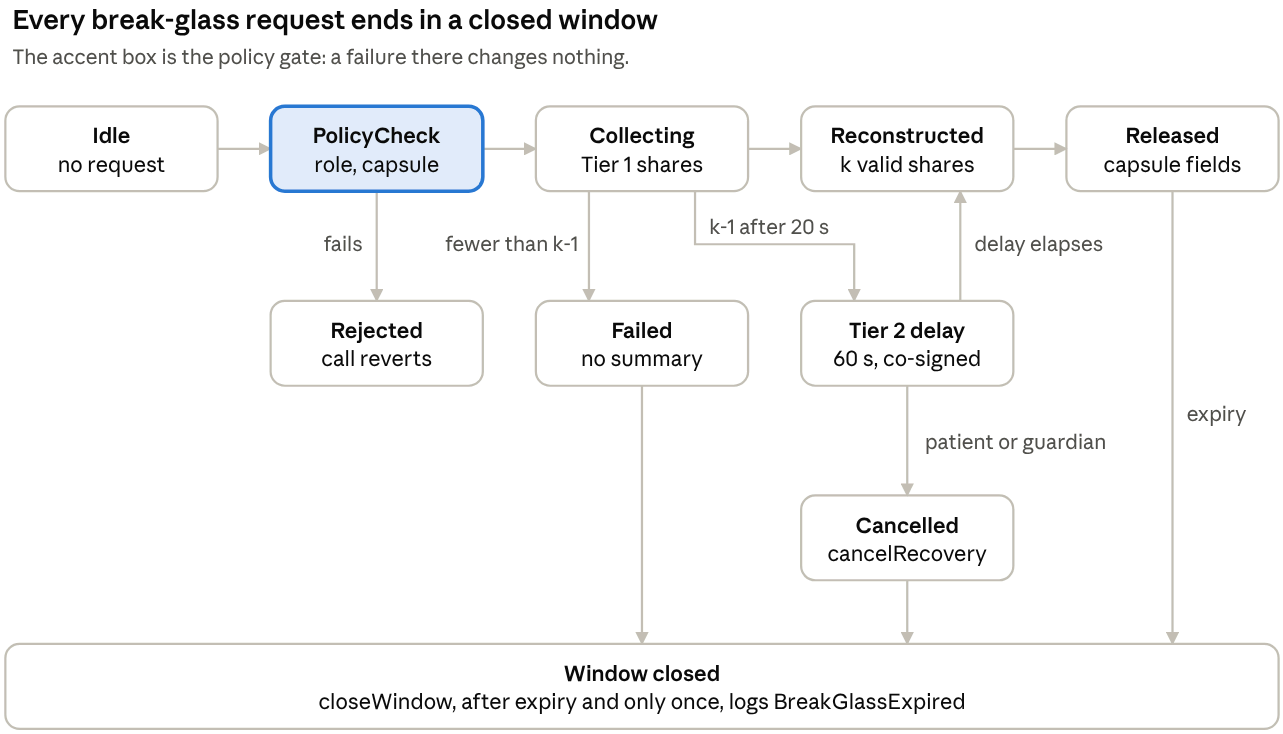
Who does what

Actor	Role in the flow
Patient	Pre-authorizes capsule fields, can turn the capsule off, can cancel during Tier 2
Responder	Holds the EmergencyResponder role, triggers the request, and reads the summary
BreakGlassCapsule	Role check, capsule check, and the window. Keeps only `windowExpiry` and `closed`
RecoveryVault	Source of truth for patientId, requester, release mode, cancelled, and tier2EndsAt. Runs the Tier 2 co-sign and delay
RecoveryCoordinator	Collects shares, checks commitments, rebuilds S, derives `capsuleKey`, returns capsule fields, wipes keys
Guardians	Scripted in the demo (decision D5). Each decrypts its bundle and re-wraps the share to the coordinator
Institutional custodian	Co-signs Tier 2. Its bundle is released after the delay
AuditLog	Records one entry per step

Before an emergency

1. The patient chooses the capsule fields on the Emergency info screen: blood type, allergies, current medications, and emergency contacts.
2. `preAuthorizeCapsule(fields)` stores the field names as hashes and bumps the version.
3. The client library encrypts the `CapsulePayload` under `capsuleKey` and registers it under the EmergencyCapsule type (errata 7). Only the ticked fields are present.
4. `revokeCapsule()` turns emergency access off.

State machine



break-glass request states

A failed role or capsule check ends in Rejected with nothing changed. Every request that opens a window ends in Window closed, whether it was released, cancelled, or failed. With k = 3 and four guardians, three or more valid shares release the summary, exactly two start Tier 2, and fewer than two fail. If the k-th valid share arrives during the Tier 2 delay, the secret is rebuilt at once and the delay is dropped (D23), a transition the drawing leaves out.

Sequence (BG-8 demo)

1. The responder calls `BreakGlassCapsule.triggerEmergencyAccess(patientId)`. The role check and the capsule pre-authorization check run first. Either failing reverts the whole call, so no state changes and no cryptography runs (T1-9, BG-5).
2. On success, in one transaction, the contract makes the requestId (errata 6), calls `RecoveryVault.setReleaseMode(requestId, patientId, requester, CapsuleOnly)`, sets `windowExpiry`, and logs BreakGlassTriggered. `setReleaseMode` is external and callable only by the capsule contract (errata 1, 2). If the AuditLog call reverts, everything reverts (AUD-2).
3. Off-chain, the coordinator receives `POST /requests` in capsule mode and reads `getCommitments`.
4. Guardians respond. Each decrypts its own ML-KEM bundle and re-wraps the share and salt to the coordinator's one-time key.
5. The coordinator checks each share against its commitment and rejects mismatches before any reconstruction.
6. With k valid shares it runs Lagrange interpolation to recover S, derives `capsuleKey`, decrypts the capsule, and returns only the pre-authorized fields. A full record is never assembled on this path (decision D1).
7. If exactly k-1 shares arrive by `COLLECTION_TIMEOUT` (20 s), Tier 2 starts. The registered custodian calls `custodianCoSign(patientId, requestId)` (scripted in the demo), which starts the 60 s `TIER2_DELAY` and logs TierEscalated. The patient or any guardian can call `cancelRecovery` during the delay, which logs RecoveryCancelled. After the delay the custodian's bundle is released through the same re-wrap and verify path, and S is rebuilt from k-1 guardian shares plus the custodian's. If fewer than k-1 guardians respond, the request fails and no summary is released.
8. At `windowExpiry` anyone can call `closeWindow(requestId)`. It succeeds only after expiry and only once, and it logs BreakGlassExpired. This holds for a released, a cancelled, and a failed request alike. The coordinator wipes its one-time key and the released capsule, and it never releases after `windowExpiry`.

Where request state lives (errata 2)

The vault accepts `setReleaseMode` only from the capsule contract, whose address the deploy script sets once through `setBreakGlassCapsule` (errata 16). `closeWindow` decides that a request is unknown by `windowExpiry != 0`, so a lost-key requestId reverts as unknown (errata 19).

Held in	Fields
RecoveryVault	patientId, requester, release mode, clientKeyHash, completed, cancelled, tier2EndsAt
BreakGlassCapsule	windowExpiry, closed
`getRequest(requestId)`	Reads the vault's fields and combines them with `windowExpiry` and `closed` into one status: None, WindowOpen, Tier2Waiting, Cancelled, or Closed. When a cancelled request is also closed, Cancelled wins

Timing

On the Tier 1 path guardians respond within seconds, so the summary is on screen with about 2.5 minutes left. On the Tier 2 path, 20 s of collection, the 60 s wait, and a few seconds to rebuild use about 90 s, which leaves about 90 s to read. The deploy script asserts that `WINDOW_DURATION` is at least twice `TIER2_DELAY`. All of these read chain time (D11), so the demo never advances the clock. These are demo settings only, because clinical validation is a PRD non-goal.

Parameter	Demo value	Meaning
`WINDOW_DURATION`	180 s	Starts at the trigger and covers the whole request
`TIER2_DELAY`	60 s	The time the patient has to cancel
`COLLECTION_TIMEOUT`	20 s	How long the coordinator waits for guardians before checking Tier 2

Identifying the patient

The responder screen scans the emergency card payload `medichain://patient/<patientId as 0x hex>`, with manual entry as a fallback. The card carries no medical data.

Limits

- Policy before cryptography is enforced by atomic revert, and the audit event cannot be skipped.
- Capsule-only release is enforced by the flow, not by separate keys, because the emergency quorum rebuilds S (known limit 5).
- The coordinator briefly holds S (known limit 8).
- Tier 2 needs k-1 guardians, because the custodian holds a single share (known limit 12). With k = 2, the custodian plus one guardian is enough.
- The responder's attestation is stored in the signed audit record and is UI-only. The contract does not enforce it (known limit 10).
- A summary already on a screen cannot be unseen (known limit 6).
- No notification reaches the patient or guardians, so a request is visible only after the patient opens the app (known limit 19, new in v1.2). The Tier 2 delay is enforced by the contract's timestamp and the coordinator, not by cryptography, because the custodian can decrypt its own bundle at any time (known limit 20, new in the second pass).

Zero-knowledge claims

An issuer commits to a claim, such as age over a threshold, and the patient later proves the claim meets a verifier's requirement without revealing the value. Each proof is tied to one request, so it cannot be replayed, reused for another patient, or made with an invented number (ZKP-1 to ZKP-4, ZKP-7). Zero-knowledge proofs cover issued claims only. Consent tokens are plain on-chain tokens, not zero-knowledge objects.

Roles

Party	Does
Patient	Grants a Claim permission naming the issuer. Holds the claim value and salt in the encrypted Patient record. Generates proofs in their client
Issuer	An account with the Doctor or Lab role. Computes the commitment and registers it
Verifier	Anyone who needs the claim. Supplies the required threshold and the request id

Party	Does
ProofGateway	Holds commitments, checks the inputs, uses up nonces, and logs outcomes
Generated verifier	Checks the Groth16 proof (`ProofVerifier.sol` , generated by snarkjs)

Issuance, proving, verification

A proof must match the issuer's commitment, the threshold, and a fresh nonce. The three phases are:

- **Issuance (once).** The patient grants a Claim token naming the issuer, with no wrapped key. The issuer commits `Poseidon(value, salt)` and calls `setCommitment` with the token. The gateway stores the commitment and issuer, emits `ClaimCommitted` , and logs ClaimIssued.
- **Proving (patient client).** Read the claim value and salt from the decrypted Patient record. Derive the nonce `keccak256(chainId, gateway, requestId) mod BN254` . Generate a Groth16 proof with snarkjs, 8 elements and 3 signals.
- **Verification (on-chain).** Check that the claim, threshold, commitment, and nonce all match. Verify the proof with the generated verifier contract. Use up the nonce, and log ProofVerified.

The value and threshold must fit in 8 bits. The claim is as of issuance.

Issuance (ZKP-7)

1. The patient issues a Claim consent token that names the issuer. A Claim token carries an empty wrapped-key entry.
2. The issuer computes `Poseidon(value, salt)` with circomlibjs. The value fits in 8 bits, and the salt is 31 random bytes.
3. The issuer calls `ProofGateway.setCommitment(patientId, claimType, commitment, tokenId)` . The gateway checks the token with ConsentRegistry (`isValid` , `getConsent`) and requires the Doctor or Lab role and a token that is unexpired and unrevoked, belongs to the patient's account, names the caller, and includes the Claim type. If a claim already exists, only its original issuer may replace it. A zero commitment, or one outside the field, reverts.

4. The gateway stores the commitment and the issuer's address, emits `ClaimCommitted`, and logs ClaimIssued.
5. The issuer places the value and salt in the patient's encrypted Patient record, as a valid FHIR complex extension *(v1.2)*.

There is no issuer screen. In the demo the seed script performs the patient's permission and the issuance.

Proving

1. The patient's client reads the value and salt from the decrypted Patient record.
2. It computes the commitment and the nonce, `keccak256(abi.encode(chainId, gatewayAddress, requestId))` reduced modulo the BN254 scalar field. The requestId is a request id or a consent token id.
3. It builds the witness and generates a Groth16 proof with snarkjs, in the browser or in Node. ProofService is the same step run as a service. The result is 8 field elements and 3 public signals.

Verification

`verifyEligibility(patientId, claimType, requestId, requiredThreshold, proof, publicSignals)` checks, in this order:

1. A claim exists for the patient and claim type.
2. `publicSignals[0]` is the `requiredThreshold` the verifier set, so a prover cannot choose its own.
3. `publicSignals[1]` is the stored commitment.
4. `publicSignals[2]` is `expectedNonce(requestId)`.
5. That nonce is unused.
6. The generated verifier accepts the proof.

On success the nonce is marked used and ProofVerified is logged. An invalid proof returns false and does not use up the nonce. `verifyBatch` runs the same checks for each proof inside one transaction. It saves transactions, not per-proof cost, because there is no aggregation. A proof is not bound to its submitter, so someone who sees a pending proof could submit it first and use up its nonce. That is acceptable for the local demo.

The circuit

The constraints are: `Poseidon(2)` of value and salt equals the commitment, `GreaterEqThan(8)` of value and threshold equals 1, and the nonce is squared so the compiler keeps it. The circuit has no output. It compiled with circom 2.2.3 to 531 constraints (253 non-linear and 278 linear), with 3 public and 2 private inputs. The public signal order is threshold, commitment, nonce.

Signal	Visibility	Meaning
value	Private	The issued claim value. Must fit in 8 bits
salt	Private	Blinds the commitment. 31 random bytes
threshold	Public	What the verifier requires. Must fit in 8 bits
commitment	Public	Must equal Poseidon(value, salt) and the commitment the issuer registered
nonce	Public	Makes the proof single-use

Build pipeline

1. Compile with circom (r1cs, wasm, sym), resolving circomlib from `node_modules`.
2. Trusted setup: Powers of Tau of power 12 (4,096 constraints, enough for 531), then a circuit-specific phase 2.
3. Generate the witness from the compiled WASM.
4. Prove with snarkjs Groth16.
5. Convert with `exportSolidityCallData` and verify through `ProofGateway.verifyEligibility`.

Revocation

`revokeClaim(patientId, claimType)` works for the patient's account or the issuer. It deletes the claim, emits `ClaimRevoked`, and logs it. A proof against a revoked claim reverts with no claim. A different issuer holding its own token can then register a new claim.

Cost and limits

- Gas figures are estimates from skeleton contracts: about 270,000 for one proof and about 513,000 for two in a batch. The ZKP-5 benchmark is the authority, and these must not be quoted as results.
- Proof generation should take well under a second. That is a target for the benchmark, not a result.
- The claim is as of issuance, and the verifier trusts the issuer.
- The trusted setup is local with a single contributor, so it is not production-grade.
- The value and the threshold must each fit in 8 bits.
- Groth16 is classical, so it sits outside the post-quantum scope.
- A diagnostic-threshold claim (ZKP-6) is a stretch that reuses the circuit for values that fit in 8 bits.

Audit trail

`AuditLog` is the single canonical on-chain feed. Only allow-listed contracts may call `log(patientId, actor, action, refId)`, and the caller passes the real actor, so the log names the person and not the relayer or a contract. If the log call reverts, the calling transaction reverts, so nothing that matters succeeds without a trace (AUD-1, AUD-2).

Which function logs which action

Action	Emitted by	refId
ConsentIssued, ConsentRevoked	ConsentRegistry	tokenId
AccessGranted, AccessDenied	ConsentRegistry (`checkAccess`, the logged transaction, never `canAccess`) *(v1.2)*	tokenId
RecordAdded	RecordRegistry	entryId
ProofVerified	ProofGateway	requestId
ClaimIssued	ProofGateway (`setCommitment`)	tokenId of the Claim token
ClaimRevoked	ProofGateway (`revokeClaim`)	claimType

Action	Emitted by	refId
BreakGlassTriggered, BreakGlassExpired	BreakGlassCapsule	requestId
RecoveryInitiated, RecoveryCompleted, RecoveryCancelled	RecoveryVault	requestId
TierEscalated	RecoveryVault (`custodianCoSign`)	requestId

Three other events sit beside `AuditLogged`: `KeysWrapped` (ConsentRegistry), and `ClaimCommitted` and `ClaimRevoked` (ProofGateway).

The signed off-chain record (AUD-3)

The chain cannot see everything. For access and recovery events the coordinator also writes a record with the same fields plus two optional ones: an `attestation` flag, which is the responder's tick that the patient cannot consent and is UI-only, and a `trustees` list (index, category, and status: waiting, responded, invalid, or timeout). The record is signed with ML-DSA-65 by a registered demo Coordinator identity and stored as an append-only JSONL file per patient (errata 9). Sentences such as "3 of 4 guardians responded" come from this record, because no on-chain event carries them.

Activity filters

The Activity screen (SCR-09) reads the `AuditLogged` events and groups them under three chips:

- **Sharing:** ConsentIssued, ConsentRevoked, AccessGranted, AccessDenied, ProofVerified, RecordAdded, ClaimIssued, ClaimRevoked.
- **Emergency:** BreakGlassTriggered, BreakGlassExpired.
- **Recovery:** RecoveryInitiated, RecoveryCompleted.
- **TierEscalated and RecoveryCancelled:** Emergency when the request's release mode is CapsuleOnly, Recovery when it is FullRecord.

Screens and the architecture they touch

Each screen uses a small, fixed set of contracts and services. This map names them, so a change to a contract shows which screens it can break. Rows marked v1.2 changed with the accepted decisions.

Screen	Uses
SCR-01 Connect and route *(v1.2)*	Wallet and network check, `accountOfOwner`, then the registration and role lookups that choose the next screen
SCR-02 Create or unlock your vault *(v1.2)*	`createAccount` through the relayer, `register`, key derivation from S, the device keystore
SCR-03 Patient home *(v1.2)*	`capsuleStatus`, `getGuardianSet` and `getCommitments`, `getRequest` and `getRequestStatus`, AuditLogged events, the consent tokens, and `cancelRecovery` from the alert
SCR-04 My records	`recordsOf`, the IPFS proxy fetch, signature check, local decrypt
SCR-05 Share	`issueConsent`, type-key wrapping, `verifyEligibility`
SCR-06 Who has access	The token list and `revokeConsent`
SCR-07 Guardians *(v1.2)*	The dealer launcher, `registerShareCommitments`, `rotateShareCommitments`, `getGuardianSet`
SCR-08 Emergency info	`preAuthorizeCapsule`, `revokeCapsule`, the capsule payload under `capsuleKey`
SCR-09 Activity	AuditLogged events and the signed off-chain records
SCR-10 Recover my access	`startRecovery`, the coordinator in full mode, `completeRecovery`
SCR-11 Emergency access	`triggerEmergencyAccess` and the coordinator in capsule mode, read through a fetch stream with the session token *(v1.2)*

Screen	Uses
SCR-12 Shared with me *(v1.2)*	`canAccess`, `checkAccess`, `tokenIdsOfGrantee`, `recordsOf`, the read path in the data layer section, claim verification
SCR-13 Hospital B viewer	The same read path, with its own demo identity. Calls only the IPFS proxy's read-only GET route (D18)
SCR-S1 to SCR-S3	Scripts with no screen: scripted guardian responses, the custodian co-sign, and the demo runner

Decisions D18 to D23

Six decisions were added after a review:

- **Hospital B read route (D18).** The IPFS proxy's `GET /ipfs/:cid` accepts the frontend and the Hospital B origin and needs no token, because it serves ciphertext only. Pinning stays frontend-only, and Hospital B calls no other local service.
- **Proof request and result (D19).** A verifier's request names the claim type, threshold, and request id. `ProofGateway` emits `EligibilityVerified` on success, so the verifier reads the result from the receipt and the screen shows the verified fact from the event. A proof share is not a consent token.
- **Capsule-mode authorization (D20).** The coordinator makes the requester sign a challenge and checks the signer against the on-chain requester. It requires CapsuleOnly mode and an open window, reads the newest capsule record through the proxy, checks hash and signature, and releases only the fields in `capsuleFields`. In this mode it decrypts the emergency capsule only, never another patient record. Saving emergency info registers the capsule before pre-authorizing it.
- **Tier 2 guards (D21).** `custodianCoSign` reverts for an unknown, cancelled, completed, or already co-signed request, so it cannot reset the delay. Known limit 20 says the delay is not cryptographic.
- **Tokens (D22).** A session token starts requests and calls the dealer and the proxy. A per-request token guards events, result, close, and shares. A share must also be wrapped to the request's one-time key.

- **Small fixes (D23).** The circuit range-checks value and threshold to 8 bits, and the gateway rejects a threshold of 256 or more. ProofService is a benchmark and seed helper, not a demo process, although the first drawing still shows it in the services band. A k-th valid share during the Tier 2 delay rebuilds S at once.